

Name: vaishnavi rajendra gayake

Subject:AIML

College name:T.b.girwalkar polytechnic college ambajogai

Branch:electronics and computer

Q1.(Data Loading & Preprocessing) Load the Heart Disease dataset.

```
import pandas as pd

df = pd.read_csv("heart.csv")

print("First 5 rows of dataset:")

print(df.head())

X = df.drop("HeartDisease", axis=1)

y = df["HeartDisease"]

categorical_columns = X.select_dtypes(include=["object"]).columns

print("\nCategorical Columns:")

print(list(categorical_columns))

X_encoded = pd.get_dummies(X, columns=categorical_columns, drop_first=True)

X_encoded = X_encoded.astype(int)

print("\nEncoded Features (X):")

print(X_encoded.head())

print("\nTarget Variable (y):")

print(y.head())

print("\nShape of X:", X_encoded.shape)

print("Shape of y:", y.shape)
```

Q2.(Train-Test Split)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.20,
    random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)
```

Q3.(Building Logistic Regression Model)

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

print("Logistic Regression model trained successfully!")
```

Q4.(Making Predictions,)

```
y_pred = model.predict(X_test)

print("First 10 Actual Values (y_test):")
print(y_test.iloc[:10].values)

print("\nFirst 10 Predicted Values (y_pred):")
print(y_pred[:10])
```

Q5.(Confusion Matrix)

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

TN, FP, FN, TP = cm.ravel()

print("Confusion Matrix:")

print(cm)

print("\nTrue Negative (TN):", TN)

print("False Positive (FP):", FP)

print("False Negative (FN):", FN)

print("True Positive (TP):", TP)
```

Q6.(Model Evaluation Metrics)

```
from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    classification_report

)

accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Q7.(Saving the Model)

```
import joblib
joblib.dump(model, "heart_model.pkl")
joblib.dump(X_encoded.columns.tolist(), "columns.pkl")
print("Model saved as heart_model.pkl")
print("Columns saved as columns.pkl")
```

Q8.(Loading & Testing Saved Model)

```
import pandas as pd
import joblib
loaded_model = joblib.load("heart_model.pkl")
columns = joblib.load("columns.pkl")
```

```

sample_data = {
    "Age": [50],
    "Sex": ["M"],
    "ChestPainType": ["ATA"],
    "RestingBP": [120],
    "Cholesterol": [200],
    "FastingBS": [0],
    "RestingECG": ["Normal"],
    "MaxHR": [150],
    "ExerciseAngina": ["N"],
    "Oldpeak": [1.0],
    "ST_Slope": ["Up"]
}

sample_df = pd.DataFrame(sample_data)
sample_encoded = pd.get_dummies(sample_df)
sample_encoded = sample_encoded.reindex(columns=columns, fill_value=0)
prediction = loaded_model.predict(sample_encoded)

if prediction[0] == 1:
    print("Prediction: Heart Disease Present")
else:
    print("Prediction: Heart Disease Not Present")

```

Q9.(Streamlit Input Interface)

```
import streamlit as st
```

```
import pandas as pd

import joblib

model = joblib.load("heart_model.pkl")

columns = joblib.load("columns.pkl")

st.set_page_config(

    page_title="Heart Disease Prediction",

    page_icon="  ",

    layout="centered"

)

st.title("  Heart Disease Prediction")

st.write("Enter patient details to predict the possibility of heart disease.")

Age = st.number_input(

    "Age",

    min_value=1,

    max_value=120,

    value=50

)

Sex = st.selectbox(

    "Sex",

    ["M", "F"]

)

ChestPainType = st.selectbox(

    "Chest Pain Type",

    ["ATA", "NAP", "ASY", "TA"]

)
```

```
RestingBP = st.number_input(  
    "Resting Blood Pressure",  
    min_value=0,  
    max_value=250,  
    value=120  
)
```

```
Cholesterol = st.number_input(  
    "Cholesterol",  
    min_value=0,  
    max_value=700,  
    value=200  
)
```

```
FastingBS = st.selectbox(  
    "Fasting Blood Sugar",  
    [0, 1]  
)
```

```
RestingECG = st.selectbox(  
    "Resting ECG",  
    ["Normal", "ST", "LVH"]  
)
```

```
MaxHR = st.number_input(  
    "Maximum Heart Rate",  
    min_value=50,  
    max_value=250,  
    value=150
```



```

)

ExerciseAngina = st.selectbox(
    "Exercise-Induced Angina",
    ["N", "Y"]
)

Oldpeak = st.number_input(
    "Oldpeak",
    min_value=-5.0,
    max_value=10.0,
    value=1.0,
    step=0.1
)

ST_Slope = st.selectbox(
    "ST Slope",
    ["Up", "Flat", "Down"]
)

if st.button("Predict"):
    input_data = pd.DataFrame({
        "Age": [Age],
        "Sex": [Sex],
        "ChestPainType": [ChestPainType],
        "RestingBP": [RestingBP],
        "Cholesterol": [Cholesterol],
        "FastingBS": [FastingBS],
        "RestingECG": [RestingECG],

```

```

    "MaxHR": [MaxHR],
    "ExerciseAngina": [ExerciseAngina],
    "Oldpeak": [Oldpeak],
    "ST_Slope": [ST_Slope]
})
input_encoded = pd.get_dummies(input_data)
input_encoded = input_encoded.reindex(
    columns=columns,
    fill_value=0
)
prediction = model.predict(input_encoded)
if prediction[0] == 1:
    st.error(" Prediction: Heart Disease Present")
else:
    st.success(" Prediction: Heart Disease Not Present")

```

Q10.(Complete Streamlit Web App)

```

import streamlit as st

import pandas as pd

import joblib

model = joblib.load("heart_model.pkl")

columns = joblib.load("columns.pkl")

st.set_page_config(

```

```
    page_title="Heart Disease Prediction",
    page_icon="",
    layout="centered"
)
st.title(" Heart Disease Prediction")
st.write("Enter the patient's details to predict heart disease.")
Age = st.number_input(
    "Age",
    min_value=1,
    max_value=120,
    value=50
)
Sex = st.selectbox(
    "Sex",
    ["M", "F"]
)
ChestPainType = st.selectbox(
    "Chest Pain Type",
    ["ATA", "NAP", "ASY", "TA"]
)
RestingBP = st.number_input(
    "Resting Blood Pressure",
    min_value=0,
    max_value=250,
    value=120
```

```
)  
Cholesterol = st.number_input(  
    "Cholesterol",  
    min_value=0,  
    max_value=700,  
    value=200  
)  
FastingBS = st.selectbox(  
    "Fasting Blood Sugar",  
    [0, 1]  
)  
RestingECG = st.selectbox(  
    "Resting ECG",  
    ["Normal", "ST", "LVH"]  
)  
MaxHR = st.number_input(  
    "Maximum Heart Rate",  
    min_value=50,  
    max_value=250,  
    value=150  
)  
ExerciseAngina = st.selectbox(  
    "Exercise Angina",  
    ["N", "Y"]  
)
```

```
Oldpeak = st.number_input(
    "Oldpeak",
    min_value=-5.0,
    max_value=10.0,
    value=1.0,
    step=0.1
)
ST_Slope = st.selectbox(
    "ST Slope",
    ["Up", "Flat", "Down"]
)
if st.button("Predict"):
    input_data = pd.DataFrame({
        "Age": [Age],
        "Sex": [Sex],
        "ChestPainType": [ChestPainType],
        "RestingBP": [RestingBP],
        "Cholesterol": [Cholesterol],
        "FastingBS": [FastingBS],
        "RestingECG": [RestingECG],
        "MaxHR": [MaxHR],
        "ExerciseAngina": [ExerciseAngina],
        "Oldpeak": [Oldpeak],
        "ST_Slope": [ST_Slope]
    })
    input_encoded = pd.get_dummies(input_data)
```

```
input_encoded = input_encoded.reindex(  
    columns=columns,  
    fill_value=0  
)  
prediction = model.predict(input_encoded)  
if prediction[0] == 1:  
    st.error(" Heart Disease: Yes")  
else:  
    st.success(" Heart Disease: No")
```